1. Why do we need architecture in any project?

Software architecture organizes the code into structured components, making the project easier to maintain, test, scale, and extend. It also reduces complexity and improves code reusability.

2. What is N-Tier Architecture?

N-Tier Architecture divides an application into separate layers, where each layer has a specific responsibility.

Common layers:

- Presentation Layer: Handles user interaction.
- Business Logic Layer: Contains business rules.
- Data Access Layer: Handles database operations.
- Database Layer: Stores application data.

3. What is Onion Architecture?

Onion Architecture organizes the application into layers around the Domain layer, which is at the center.

Its main principle is Dependency Inversion: outer layers depend on inner layers, not the opposite.

Main layers:

- Domain
- Application
- Infrastructure
- Presentation

4. Is LINQ slow in execution?

No, LINQ is not necessarily slow. Its performance depends on how it is used.

- Deferred Execution: The query executes only when the results are requested, such as using `foreach`.
- Eager Execution: The query executes immediately, such as using `ToList()`, `Count()`, or `First()`.

Deferred execution can improve performance by avoiding unnecessary operations.

Microsoft Learn+1

5. Give three use cases for asynchronous programming.

1. API Calls: Calling external APIs without blocking the application.
2. Database Operations: Executing database queries asynchronously.
3. File Operations: Reading or writing large files without blocking the main thread.

6. What is the difference between Thread and Task?

Thread	Task
Represents an actual thread of execution.	Represents an asynchronous operation.
Managed directly by the developer.	Usually managed by the Task Scheduler and ThreadPool.
Has more overhead.	Generally lightweight.
Used for low-level thread control.	Used for modern asynchronous programming.

In short: A Thread is an execution resource, while a Task represents work that may use a thread to execute.

7. What are the new features in .NET 9 for working with Generic Types?

.NET 9 uses C# 13, which introduces:

- `allows ref struct`: Allows `ref struct` types, such as `Span<T>`, to be used as generic type arguments.
- `ref struct` Interfaces: Allows `ref struct` types to implement interfaces.
- `params` Collections: Allows generic methods to accept different collection types through `params`.

Note: Generic Math, which allows mathematical operations on generic types using interfaces like `INumber<T>`, was introduced in .NET 7, not .NET 9.

Microsoft Learn+2

8. What are the built-in Delegate types in C#?

A Delegate is a type-safe reference to a method. C# provides several built-in delegate types:

1. Action: Represents a method that takes zero or more parameters and returns nothing (`void`).
2. Func: Represents a method that takes zero or more parameters and returns a value. The last generic type represents the return type.
3. Predicate: Represents a method that takes one parameter and returns a Boolean value (`true` or `false`).
4. EventHandler: Used to handle events and notify subscribers when an event occurs.